	**UNIDADES TECNOLÓGICAS DE SANTANDER** Nuevas Tecnologías de Desarrollo (TSI601) **TALLER: Patrones de diseño e inyección de dependencias** *Cómo organiza su código la industria del software* **Nombre:** _____ **Fecha:** _____

Introducción

Durante el curso se ha organizado el código con el patrón Modelo-Vista-Controlador. En esta sesión se estudian dos conceptos que usan las empresas de software para que sus sistemas sean ordenados, mantenibles y fáciles de probar: los patrones de diseño y la inyección de dependencias.

1. ¿Qué es un patrón de diseño?

Un patrón de diseño es una solución ya probada para un problema que se repite al construir software. En lugar de inventar una solución desde cero cada vez, se reutiliza una forma de organizar el código que la industria ya validó durante años.

El patrón MVC, usado durante todo el curso, es un ejemplo: separa los datos (Modelo), la presentación (Vista) y la coordinación (Controlador). Frameworks como Laravel (PHP), Django (Python) y Angular (JavaScript) están construidos sobre este patrón.

> *Un patrón no es un lenguaje ni una herramienta: es una forma recomendada de estructurar el código para resolver un problema frecuente.*

Patrones más usados en la industria

A continuación, los patrones más comunes, con un caso real de uso:

Patrón	Qué resuelve	Caso real
MVC	Separar datos, presentación y lógica.	Laravel, Django y Angular organizan sus aplicaciones así.
Singleton	Garantizar una única instancia de un recurso compartido.	La conexión a la base de datos de un sistema: se abre una sola y todos la usan, en lugar de abrir una por cada consulta.
Factory (Fábrica)	Crear objetos sin decidir manualmente de qué tipo, según el contexto.	Un sistema de pagos que, según el país, crea el objeto "pago con PSE", "pago con tarjeta" o "pago con Nequi".
Observer (Observador)	Avisar automáticamente a varias partes cuando algo cambia.	YouTube: al suscribirte a un canal, el sistema te notifica cada video nuevo. Lo mismo con las notificaciones de una red social.

	**UNIDADES TECNOLÓGICAS DE SANTANDER** Nuevas Tecnologías de Desarrollo (TSI601) **TALLER: Patrones de diseño e inyección de dependencias** *Cómo organiza su código la industria del software* **Nombre:** _____ **Fecha:** _____

2. ¿Qué es la inyección de dependencias?

Una dependencia es algo que una parte del programa necesita para funcionar. En el curso, el Controlador necesita el Modelo para pedirle los datos: el Modelo es una dependencia del Controlador.

Inyectar una dependencia significa entregarle a una parte del programa lo que necesita desde afuera, en lugar de que ella lo cree por dentro. Así, se puede cambiar esa dependencia sin modificar la parte que la usa.

¿Quién lo usa y por qué?

La inyección de dependencias es la base de los frameworks profesionales más usados:

- Angular (Google) y Spring (Java, el que menciona el plan de la asignatura) están construidos sobre inyección de dependencias: es su mecanismo central.
- .NET (Microsoft) y NestJS la usan de la misma forma.

Se usa porque resuelve problemas reales en sistemas grandes:

- Mantenimiento: un sistema que mantienen muchas personas durante años. Con dependencias desacopladas, cambiar una parte no rompe las demás.
- Pruebas: se le puede entregar al Controlador un Modelo "de prueba" con datos falsos, para verificarlo sin tocar la base de datos real. (Esto es la base del tema del tercer corte: pruebas y TDD.)
- Cambios sin reescribir: cambiar de un servicio de datos a otro, o de una base de datos a otra, sin modificar quien los usa.

3. Ejemplo en código

Las dos formas de dar al Controlador su dependencia (el Modelo):

Sin inyección (el Controlador crea el Modelo por dentro):

```
function iniciarControlador() {
  const modelo = new Modelo();   // lo crea el mismo
  // ... usa modelo
}
```

Con inyección (el Modelo se recibe desde afuera):

```
function iniciarControlador(modelo) {   // se lo entregan
```

	**UNIDADES TECNOLÓGICAS DE SANTANDER** Nuevas Tecnologías de Desarrollo (TSI601) **TALLER: Patrones de diseño e inyección de dependencias** *Cómo organiza su código la industria del software* **Nombre:** _____ **Fecha:** _____

```
  // ... usa modelo
}

// Al usarlo, se decide qué modelo entregar:
iniciarControlador(modeloSimulado);
// o, en el futuro, sin cambiar el Controlador:
iniciarControlador(modeloBaseDeDatos);
```

> *El mismo Controlador funciona con cualquier Modelo que se le entregue. Por eso el código queda reutilizable y fácil de probar.*

Actividad 1 — ¿Qué patrón se usa?

Para cada caso real, indicar qué patrón de diseño se está aplicando (MVC, Singleton, Factory u Observer):

a) Un sistema abre una sola conexión a la base de datos y todos la comparten: _____

b) YouTube avisa a los suscriptores cuando hay un video nuevo: _____

c) Una app separa los datos, la pantalla y la lógica en partes distintas: _____

d) Un sistema de pagos crea el objeto adecuado según el método elegido: _____

Actividad 2 — Patrón o no patrón

Indicar si cada afirmación describe un PATRÓN DE DISEÑO (P) o NO (N), y justificar brevemente:

a) Separar el código en Modelo, Vista y Controlador. (___)

b) Escribir todo el programa en un único archivo revuelto. (___)

c) Mantener una sola instancia de la configuración (Singleton). (___)

d) Escribir los nombres de variables en inglés. (___)

Actividad 3 — Identificar dependencias

Para cada componente, indicar qué dependencia necesita para funcionar:

a) Un Controlador que muestra una lista de productos necesita: _____

b) Una Vista que muestra el clima necesita los datos de: _____

c) Un sistema de login necesita, para verificar las credenciales, el: _____

	**UNIDADES TECNOLÓGICAS DE SANTANDER** Nuevas Tecnologías de Desarrollo (TSI601) **TALLER: Patrones de diseño e inyección de dependencias** *Cómo organiza su código la industria del software* **Nombre:** _____ **Fecha:** _____

d) Un sistema que envía facturas por correo necesita un: _____

Actividad 4 — Inyección de dependencias en el código

Observando el ejemplo del punto 3, responder:

1. ¿Qué diferencia hay entre que el Controlador cree el Modelo por dentro y que lo reciba desde afuera?

---

2. En el buscador de países de la práctica anterior se usó un servicio simulado. Si mañana se cambiara por un servicio real, ¿por qué la inyección de dependencias facilitaría ese cambio?

Actividad 5 — Aplicado al proyecto

Pensando en el sistema de la empresa del proyecto de aula:

1. ¿Qué dependencia necesita el Controlador del sistema para funcionar?

---

2. ¿Qué patrón o patrones de los vistos aplicaría el sistema y por qué?

Preguntas de análisis

- 1. Con sus palabras, ¿qué es un patrón de diseño? Dé un ejemplo real.

- 2. ¿Qué es una dependencia y qué significa inyectarla?

- 3. ¿Por qué los frameworks profesionales (Angular, Spring) usan inyección de dependencias?